Prompt Generation Frameworks

Here are some of the popular frameworks for prompt generation, optimized to minimize token usage while delivering optimal results. Each framework has its strengths, so choose based on the specific needs of your task or communication. Which one do you think fits your current needs best?

- **R.A.C.E**

When to use: For detailed action plans with context

R | Role: You are a C# expert with proficiency in database management.

A | Action: Create a C# code template that retrieves data from an On-Premises SQL Server in configurable date-based batches and updates a table in an Azure SQL Managed Instance using the retrieved data.

C | Context: Team has stale data on Azure SQL MI, we want migrate data from On-Premises SQL Server to an Azure SQL Managed Instance. To ensure efficiency and manageability, data will be processed in batches defined by configurable date ranges. The C# code should incorporate the necessary SQL queries and logic to facilitate this batch processing.

E | Expectation: The expected outcome is a C# code template that:

Connects to the On-Premises SQL Server.
Reads data in batches based on configurable date ranges.
Updates the Azure SQL Managed Instance table with the retrieved data.
Implements error handling and logging for the migration process.

Prompt for one of the use XPG use cases using R.A.C.E framework:

> 📝 "You are a C# expert proficient in database management. Your task is to generate a C# code template that reads data from an On-Premises SQL Server in configurable date-based batches and updates an Azure SQL Managed Instance table using the retrieved data.
>
> Context: Team has stale data on Azure SQL MI, we want migrate data from an On-Premises SQL Server to Azure SQL Managed Instance. The migration process needs to be efficient and manageable, hence the data is processed in batches based on configurable

date ranges. The C# code should include the necessary SQL queries and logic to handle this batch processing.

Expectation: The expected outcome is a C# code template that:

1. Connects to the On-Premises SQL Server.
2. Reads data in batches based on configurable date ranges.
3. Connects to the Azure SQL MI Server.
4. Updates the Azure SQL Managed Instance table with the retrieved data.
5. Ensures error handling and logging for the migration process, including logging errors to the console."

- **A.P.E**
  When to use: For clear, concise task definitions.
  - **A | Action:** Define the job or activity.
  - **P | Purpose:** Discuss the goal.
  - **E | Expectation:** State the desired outcome.
  - **Example**:
    - Action: Design a user authentication module.
    - Purpose: Ensure secure access to the application.
    - Expectation: Implement a module that supports login, registration, and password recovery.

- **T.A.G**
  When to use: For step-by-step task instructions.
  - **T | Task:** Define the task.
  - **A | Action:** Describe the steps.
  - **G | Goal:** Explain the end goal.
  - **Example**:
    - Task: Develop a RESTful API for the application.
    - Action:
      1. Set up the server environment.
      2. Define the API endpoints.
      3. Implement the endpoints.
      4. Test the API.

- Goal: Provide a functional API for client-server communication.

Prompt used for Creation of Documentation of XP.Templates using T.A.G framework for Alchemy

Task:
Create documentation for the project, including general documentation, UML diagrams, and a stakeholder page.

Action:
Describe the steps: Analyze the code in the project to understand its functionality.
Create general documentation: Document what the code does.
Store the documentation in markdown files within the /docs/ folder.
Ensure the main file is named index.md.
Create UML diagrams: Identify critical areas of the code.
Use MermaidJS to create diagrams for these areas.
Store each diagram in its own file within a dedicated folder.
Include an index.md file in the diagrams folder with links to all diagrams.
Add a stakeholder page:
Target non-technical stakeholders.
Describe the project with minimal jargon.
Explain the code at a high level and briefly describe major components.
Include an example of module usage.

Goal:
Provide comprehensive documentation that helps both technical and non-technical stakeholders understand the project. Ensure the documentation is easy to navigate and understand, facilitating better communication between developers and stakeholders.
Store all documentation as markdown files in the /docs/ folder, compatible with Backstage TechDocs.
Use MermaidJS for diagrams and place each in its own file within a dedicated folder.
Ensure the stakeholder page is clear and jargon-free, explaining the project at a high level and including an example of module usage.

Forbidden Actions
Do not include full copies of code as examples.
Avoid trying to sell the value or impact of the code in the stakeholder page.
Do not place markdown files inside a category folder if they are the only file in that folder.

Output: Alchemy | xplatform templates | General | Overview

- **E.R.A**

  When to use: For setting clear expectations and roles.

  - **E | Expectation:** Describe the desired result.
  - **R | Role:** Specify ChatGPT's role.
  - **A | Action:** Specify needed actions.
  - **Example**:
    - Expectation: Achieve 90% test coverage for the new feature.
    - Role: Assist in writing unit tests.
    - Action: Write and run unit tests for all new functions

- **R.I.S.E**

  When to use: For structured requests with clear inputs and scenarios.

  - **R | Request:** Specify ChatGPT's role.
  - **I | Input:** Provide necessary information.
  - **S | Scenario:** Detail the steps.
  - **E | Expectation:** Describe the result.
  - **Example**:
    - Request: Generate test cases.
    - Input: Provide the feature specifications.
    - Scenario: Write test cases based on the provided specifications.
    - Expectation: Create comprehensive test cases that cover all scenarios.

- **C.A.R.E**

  When to use: For comprehensive task descriptions with examples.

  - **C | Context:** Set the stage.
  - **A | Action:** Describe the task.
  - **R | Result:** Describe the outcome.
  - **E | Example:** Give an illustration.
  - **Example**:
    - Context: We are developing a new feature for our app.
    - Action: Implement the feature and write unit tests.
    - Result: The feature is fully functional and tested.

        - Example: Implementing a new search functionality with autocomplete.

- **C.O.A.S.T**

  When to use: For goal-oriented task planning like setting up a project timeline

  - **C | Context:** Set the stage.
  - **O | Objective:** Describe the goal.
  - **A | Actions:** Explain needed steps.
  - **S | Steps:** Describe the situation.
  - **T | Task:** Outline the task.
  - **Example**:
    - Context: We need to improve the app's performance.
    - Objective: Reduce load times by 50%.
    - Actions:
      1. Identify performance bottlenecks.
      2. Optimize the code.
      3. Test the improvements.
    - Steps: Analyze the current performance metrics.
    - Task: Implement code optimizations.

- **T.R.A.C.E**

  When to use: For tasks requiring detailed context and examples like developing a new product feature.

  - **T | Task:** Define the task.
  - **R | Role:** Describe the need.
  - **A | Action:** State the required action.
  - **C | Context:** Provide context or situation.
  - **E | Expectation:** Illustrate with an example.
  - **Example:**
    - Task: Conduct a code review.
    - Role: Code reviewer.
    - Action: Review the code for quality and adherence to standards.
    - Context: The code is part of a critical feature.
    - Expectation: Ensure the code is maintainable and efficient.

- **R.O.S.E.S**

  When to use: For solution-focused task planning like problem-solving sessions or brainstorming solutions

  - **R | Role:** Specify ChatGPT's role.
  - **O | Objective:** State the goal or aim.
  - **S | Steps:** Describe the situation.
  - **E | Expected Solution:** Define the outcome.
  - **S | Scenario:** Ask for actions needed to reach the solution.
  - **Example**:
    - Role: Test case generator.
    - Objective: Ensure the new feature is thoroughly tested.
    - Steps: Write test cases based on the feature requirements.
    - Expected Solution: Comprehensive test coverage.
    - Scenario: Provide detailed steps for writing the test cases.